

System-Level Performance Analysis of Visual SLAM and Reinforcement Learning Integration for Quadruped Navigation

You-Hao Li, Jaesik Jeong*

Abstract: This paper presents a system-level study on the real-time integration of visual SLAM, reinforcement learning (RL)-based locomotion, and dense 3D reconstruction for quadruped robot navigation in simulated indoor environments. A ROS2-based hierarchical framework is developed, integrating ORB-SLAM3 for pose estimation and a PPO-trained RL policy for navigation, alongside an auxiliary Open3D-based TSDF pipeline for dense mapping evaluation. Compared with studies that evaluate perception and control modules separately, this study examines observed system-level interactions under concurrent execution, particularly asynchronous perception–control loops, computational resource contention, and cross-module error propagation. The results suggest that delayed and intermittent SLAM pose updates can introduce temporally inconsistent observations for the RL policy, which is associated with degraded locomotion performance in constrained environments. Dense reconstruction results further show visible geometric distortion when TSDF mapping is performed using SLAM-estimated poses affected by drift. Experiments in Isaac Lab across rooms, warehouse-like scenes, and narrow corridors show that navigation and system performance are affected by environmental complexity and system load. These results provide an empirical characterization of system-level interactions between SLAM, RL, and dense mapping modules under simulated conditions. The experiments indicate that dense mapping introduces substantial memory overhead, while delayed and intermittent SLAM pose updates may degrade navigation performance under increased system load. The findings highlight several observed bottlenecks and provide practical insights for the design of vision-based quadruped navigation systems operating under computational constraints.

Keywords: quadruped robot, visual SLAM, reinforcement learning, vision-based navigation, RGB-D sensing, 3D mapping, simulation.

1. INTRODUCTION

In disaster scenarios such as earthquakes, industrial accidents, or building collapses, indoor environments often become highly unstructured and hazardous. Debris, damaged infrastructure, narrow passages, and unstable terrain pose significant challenges for both human rescuers and autonomous robotic systems. Rapid exploration and reliable environment reconstruction are therefore essential for situational awareness and search-and-rescue operations [1].

Recent advances in mobile robotics have explored different platforms for operating in such environments. Wheeled and tracked robots provide stable locomotion on structured terrains and have been widely adopted in industrial inspection and rescue applications. However, their mobility is fundamentally limited in unstructured environments. Aerial robots enable fast coverage and mapping capabilities, but suffer from limited endurance and lack of physical interaction with the environment. In contrast, quadruped robots have emerged as a promising alternative due to their superior adaptability over uneven

and obstacle-rich terrain [2]. Recent works further demonstrate their adaptability in complex environments [3, 4].

In parallel, vision-based perception systems have become increasingly popular in robotic navigation. Mapless navigation using deep reinforcement learning has been shown effective for continuous robot control without explicit mapping [5]. Visual simultaneous localization and mapping (VSLAM) enables robots to estimate pose and reconstruct environments using low-cost RGB-D sensors. Representative systems such as ORB-SLAM3 demonstrate accurate pose estimation and mapping in structured environments [6]. However, these methods remain sensitive to motion blur, illumination changes, and repetitive textures, which can degrade performance in real-world deployments, as widely discussed in the SLAM literature [7].

Meanwhile, reinforcement learning (RL) has emerged as a powerful paradigm for legged locomotion and control. By formulating locomotion as a Markov decision process, RL-based methods enable quadruped robots to learn adaptive gait policies without explicit kinematic modeling [8–10]. These approaches have shown strong performance

You-Hao Li and Jaesik Jeong are with the Department of Artificial Intelligence, Tamkang University, New Taipei City 251301, Taiwan (e-mail: 0819mhao@gmail.com, 167030@o365.tku.edu.tw).

* Corresponding author.

in simulation and partial success in real-world transfer, particularly for stable walking and obstacle avoidance behaviors [2, 9].

Furthermore, while traditional ROS2 navigation stacks (e.g., Nav2) excel in structured 2D environments, they fundamentally assume a flat, traversable manifold. This assumption often fails in the highly unstructured, post-disaster scenarios targeted in this study. Therefore, we deliberately bypass traditional global planners, relying instead on a purely RL-based hierarchical policy to handle complex, non-planar locomotion and navigate without relying on pre-computed 2D occupancy grids.

Despite these advances, achieving robust autonomous navigation through the integration of VSLAM and RL remains an open challenge. Existing robotic systems typically design perception, mapping, and control modules independently, assuming that system integration can be achieved without significant performance degradation. However, in practice, such assumptions often do not hold [11]. In particular, VSLAM and RL operate under fundamentally different system assumptions: SLAM relies on asynchronous geometric estimation with variable update rates, whereas RL policies assume stable, low-latency, and consistent state representations. This mismatch introduces system-level coupling effects, including temporal inconsistency, computational resource contention, and error propagation in closed-loop control [12].

These issues become more pronounced in real-time navigation systems, where perception and control are tightly coupled under limited computational resources and distributed communication latency [11]. As a result, even well-performing individual modules may fail to produce stable navigation behavior when integrated, especially under unseen or distribution-shifted environments.

To address these challenges, this study proposes a navigation framework based on ROS2 and vision for quadruped robots that integrates visual SLAM, reinforcement learning-based locomotion control, and dense mapping within a unified system architecture. The framework is implemented and evaluated in the Isaac Lab simulation environment under environments with varying structural complexity, including structured warehouse-like scenes and narrow corridor settings [13].

The main contributions of this paper are:

- 1) **A system-level evaluation of integrated visual SLAM, RL-based locomotion, and dense mapping** within a unified ROS2 framework, highlighting the practical challenges associated with concurrent execution, including asynchronous perception-control loops, computational resource contention, and cross-module performance interactions.
- 2) **An empirical analysis of the impact of SLAM pose instability** on RL-based navigation performance, showing that delayed and intermittent pose

updates are associated with reduced control frequency and degraded navigation performance under increasing system load.

- 3) **An empirical characterization of environment-dependent failure modes and computational bottlenecks** observed during integrated system operation, including increasing memory consumption associated with TSDF-based dense mapping and reduced navigation performance in narrow corridor environments.
- 4) **Design guidelines derived from empirical findings**, including the potential benefit of asynchronous pose buffering for feature-poor regions observed in Map A, modular resource isolation to reduce TSDF-related FPS degradation, and broader narrow-space training data to mitigate reactive circular behaviors observed in Map C.

2. RELATED WORK

2.1. Quadruped Robots for Exploration

Gibson Environment enables realistic 3D indoor navigation simulation for embodied agents [14]. Mobile robots have been extensively studied for exploration and search-and-rescue tasks in hazardous environments. Wheeled and tracked robots provide stable locomotion on structured terrains but often struggle in highly unstructured environments such as collapsed buildings and narrow passages [1]. In contrast, quadruped robots offer superior mobility and terrain adaptability due to their legged locomotion capabilities. Platforms such as ANYmal demonstrate robust traversal performance over uneven and obstacle-rich terrain [2], making quadruped robots promising candidates for autonomous exploration tasks.

2.2. Visual SLAM for Robotic Navigation

Visual simultaneous localization and mapping (VSLAM) has become a widely adopted approach for robotic localization and environment reconstruction using low-cost visual sensors. ORB-SLAM3 achieves accurate localization and multi-map tracking using RGB-D and visual-inertial inputs [6]. However, visual SLAM systems remain sensitive to environmental conditions such as illumination variation, motion blur, and repetitive textures [7, 15, 16]. These limitations become more severe in legged robotic systems due to body oscillation and unstable camera motion during locomotion.

ORB-SLAM3 is a feature-based SLAM system that estimates camera pose and reconstructs a sparse map of the environment by extracting and tracking ORB keypoints across consecutive frames [6]. The system maintains a keyframe database and performs motion estimation through feature matching and geometric optimization. In parallel, a pose graph is continuously refined using bundle

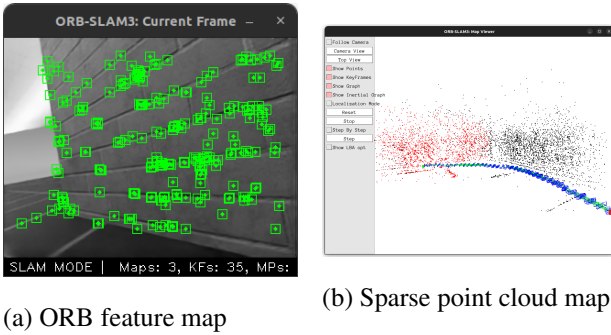


Fig. 1. ORB-SLAM3 feature extraction and mapping results. (a) Detected ORB keypoints used for feature matching. (b) Reconstructed sparse 3D map generated from visual SLAM.

adjustment to improve global consistency. Loop closure detection is employed to reduce accumulated drift by re-aligning revisited places within the map.

In this work, ORB-SLAM3 operates on synchronized RGB-D inputs and outputs real-time camera poses that are used for high-level navigation. Figure 1 illustrates the ORB feature extraction process and the corresponding sparse point cloud generated during SLAM execution. The feature map highlights the distribution of detected keypoints, while the reconstructed point cloud represents the estimated geometric structure of the environment used for localization.

2.3. Reinforcement Learning for Legged Locomotion

Reinforcement learning has recently shown strong performance in legged locomotion control. PPO-based optimization methods enable quadruped robots to learn adaptive locomotion policies directly from environmental interaction [8]. Prior studies demonstrate that RL-based policies can generate stable walking and obstacle avoidance behaviors in complex simulated environments [9]. Nevertheless, policy robustness remains highly dependent on training environment diversity, and performance degradation is commonly observed under unseen environmental geometries.

2.4. Integrated Robotic Navigation Systems

Active SLAM methods have been explored to jointly learn exploration and mapping strategies in unknown environments [17]. Although substantial progress has been made in SLAM and locomotion independently, integrating perception and learned control into a unified robotic framework remains challenging. ROS2 is adopted as the system middleware due to its modular publish-subscribe architecture and support for distributed real-time communication, making it well-suited for multi-module robotic systems [11, 12]. However, combining SLAM, dense map-

ping, and RL-based control introduces system-level issues including communication latency, computational resource contention, synchronization delay, and perception-control mismatch. Existing studies primarily focus on improving individual modules, while relatively limited attention has been given to analyzing the practical integration challenges and failure modes that arise in concurrently executed robotic systems.

3. SYSTEM ARCHITECTURE

Simulation platforms such as Habitat provide large-scale embodied AI environments for navigation research [18]. The proposed framework adopts a hierarchical vision-based navigation architecture integrating visual SLAM, reinforcement learning-based locomotion, and dense mapping within a unified ROS2-based robotic system (illustrated in Fig. 2). The system is designed as a decoupled multi-module pipeline, where perception, decision-making, and control are executed in parallel but synchronized through shared sensory streams and communication middleware.

The robot is equipped with an RGB-D camera as the primary sensing modality. To maintain a low-cost and highly deployable system architecture suitable for expendable search-and-rescue quadrupeds, this study deliberately restricts the perception module to pure visual SLAM (RGB-D) without high-fidelity IMU sensor fusion. By adopting this hardware constraint, a key objective is to expose the failure boundaries and error propagation of such cost-constrained systems under dynamic locomotion. The RGB stream is utilized by the Visual SLAM module (ORB-SLAM3) for pose estimation, while depth observations are processed in parallel for both locomotion control and dense environment reconstruction. This design enables independent optimization of geometric localization and depth-based perception.

Visual SLAM provides global pose estimates that are transmitted exclusively to the high-level navigation module. The high-level policy is responsible for generating navigation intentions, such as directional velocity commands, based on the estimated robot pose. Importantly, the visual SLAM module is not directly coupled to the low-level locomotion policy, allowing the control system to remain robust to potential SLAM tracking instability.

The low-level locomotion controller is implemented as a reinforcement learning policy trained in the Isaac Lab simulation environment. It receives proprioceptive states, depth-based obstacle information, and velocity commands from the high-level policy. The locomotion task is formulated as a Markov Decision Process (MDP), where the policy outputs joint-level commands to achieve stable quadrupedal motion. The reward function combines forward progress, stability constraints, collision avoidance, and torque regularization.

In parallel, dense mapping is performed using the TSDF-based pipeline [19]. Dense mapping in this project is evaluated in a decoupled manner using simulator ground-truth poses instead of SLAM-estimated poses, following classical TSDF-based volumetric reconstruction approaches such as KinectFusion [20] and later extensions [19]. This design was adopted to isolate reconstruction quality from SLAM drift and to reduce the influence of tracking degradation and computational load during reconstruction evaluation. This module operates independently from the SLAM pipeline and directly fuses RGB-D data to reconstruct the environment. The decoupled design allows the evaluation of reconstruction quality without being affected by pose estimation drift or navigation instability.

Prior evaluations of ROS2 highlight its communication overhead and real-time constraints in robotic systems [21]. ROS2 is used as the communication middleware to coordinate data exchange between modules. Sensor streams, pose estimates, velocity commands, and joint actions are transmitted through ROS2 topics. Although the system operates in a modular fashion, it introduces non-negligible system-level interactions such as communication latency, synchronization delays, and computational resource contention, which may affect real-time navigation stability.

It should be noted that dense mapping is not used as a feedback signal for locomotion control. It is evaluated primarily as an auxiliary reconstruction module and as a concurrent computational load source. The ground-truth-pose reconstruction provides a controlled reference, while the SLAM-pose reconstruction illustrates the sensitivity of TSDF mapping to pose drift. This decoupled evaluation allows us to isolate reconstruction quality from localization error, but we acknowledge it does not fully capture all degradation effects that would occur in a fully closed-loop mapping-control system.

4. INTEGRATION CHALLENGES

Although the proposed framework adopts a modular architecture, integrating visual SLAM, reinforcement learning-based control, and dense mapping within a unified ROS2 system introduces several system-level challenges that significantly affect real-time performance.

- 1) **Computational Resource Contention:** The simultaneous execution of ORB-SLAM3, reinforcement learning inference, and TSDF-based dense mapping leads to significant competition for CPU, memory and disk resources. SLAM requires continuous feature extraction and optimization, while the low-level policy demands high-frequency inference for stable locomotion control. Meanwhile, dense mapping continuously fuses point cloud data, resulting in additional memory overhead. This resource contention of-

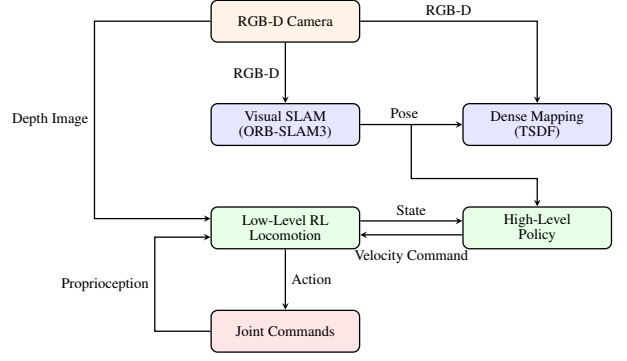


Fig. 2. Proposed hierarchical navigation system architecture. The framework integrates ORB-SLAM3 for pose estimation, a PPO-trained high-level navigation policy, a low-level RL locomotion controller, and a TSDF-based dense mapping module within a unified ROS2 communication infrastructure.

ten leads to reduced system throughput and unstable real-time performance.

- 2) **Temporal Asynchrony in Multi-Rate Systems:** The system integrates modules operating at different update frequencies, including visual SLAM, depth sensing, and locomotion control. This multi-rate design introduces temporal misalignment between perception and control loops. As a result, the robot may execute control actions based on outdated pose or depth information, which negatively affects navigation stability, particularly in narrow or cluttered environments.
- 3) **Perception–Action Coupling Instability:** Visual SLAM provides delayed and occasionally intermittent pose estimates, while the reinforcement learning policy was trained with relatively consistent and low-latency observation inputs. This mismatch can introduce temporally inconsistent observations for the learned policy, which is associated with oscillatory behaviors, suboptimal velocity commands, and degraded navigation performance under distribution-shifted environments.
- 4) **Dynamic Locomotion and Observation Timing Challenges:** In physical deployments, quadruped locomotion can introduce high-frequency body motion and camera vibration, which may degrade visual feature extraction. Since the Isaac Lab simulation used in this study does not explicitly model full physical sensor noise or realistic camera vibration, these effects are discussed as deployment-related considerations rather than experimentally verified results. In the present simulation, algorithmic delay and spatial discretization in depth processing during dynamic gaits may affect the temporal consistency of observations available to the locomotion policy.

5) **Memory Overhead in Dense Mapping:** The TSDF-based reconstruction pipeline incrementally accumulates volumetric representations of the environment. As exploration progresses, memory usage increases significantly, leading to potential computational bottlenecks and delayed map updates. Although this module is decoupled from the control pipeline, it contributes to overall system resource consumption.

5. EXPERIMENTAL SETUP

High-performance GPU-based simulators such as Isaac Gym enable massively parallel reinforcement learning for robotic control [22]. The proposed hierarchical navigation framework is evaluated in the Isaac Lab simulation environment, which provides a physics-based platform for quadruped locomotion and navigation experiments. All simulation processes, ROS2 module integration, and dense mapping evaluations were executed on a high-performance workstation equipped with an AMD Ryzen 5 9600X CPU, 64 GB of RAM, an NVIDIA RTX 4080 SUPER GPU, and a high-speed Gen4 NVMe SSD (Kioxia Exceria Plus G3). Notably, evaluating the framework on such robust hardware allows us to isolate architectural bottlenecks from hardware limitations; the observed system degradation suggests that multi-module resource contention remains important even on relatively capable hardware, implying that deployment on resource-constrained edge devices would likely require additional optimization and module decoupling.

The system is divided into two layers: a low-level locomotion controller and a high-level navigation module.

5.1. Low-Level Locomotion Policy Configuration

The low-level locomotion controller is trained in the Isaac Lab simulation environment using reinforcement learning. The policy operates at the joint-control level and is responsible for generating stable quadrupedal locomotion behaviors under different terrain conditions. Table 1 summarizes the environment configuration.

The observation space consists of proprioceptive robot states and visual depth features. Table 2 summarizes the full observation space of the low-level policy. The observation vector has a total dimension of 353, combining proprioceptive states and processed depth features.

The action space consists of 12-dimensional joint position commands corresponding to the target angles for all joints of the quadruped robot. Actions are scaled by a factor of 0.2 and applied as offsets from the default joint configuration at 25 Hz.

The reward function contains both locomotion rewards and penalty terms. Positive rewards include velocity tracking, gait synchronization, foot clearance, forward progress, and open-space seeking rewards. Penalty terms

Table 1. Low-Level Environment Configuration

Parameter	Value
Simulation timestep (dt)	0.002 s
Control decimation	20
Control frequency	25 Hz
Episode length	20 s
Camera resolution (raw)	320×240
Camera resolution (training)	15×20 (after pooling)
Depth update period	0.04 s
Depth clipping range	0.1 – 10.0 m
Linear velocity range	0.0 – 1.0 m/s
Angular velocity range	−0.4 – 0.4 rad/s
Terrain type	Flat + Random rough terrain
Number of environments	4

Table 2. Low-Level Policy Observation Space

Observation	Dim	Description
Base linear velocity	3	v_x, v_y, v_z in body frame
Base angular velocity	3	$\omega_x, \omega_y, \omega_z$
Projected gravity	3	Gravity vector in body frame
Joint positions	12	Current joint angles
Joint velocities	12	Current joint angular velocities
Actions	12	Previous joint position commands
Velocity commands	3	Target v_x, v_y, ω_z
Depth image	300	320×240 depth map resized to 60×80 , downsampled to 15×20 via average pooling (stride 4), normalized to $[-1, 1]$
Depth statistics	5	Left/center/right mean depth, frontal minimum, left-right balance
Total	353	

include body instability, excessive joint motion, foot slipping, abrupt action changes, wall proximity penalties, and illegal body contacts. The reward function is formally defined as:

$$r = \sum_i w_i \cdot r_i + \sum_j w_j \cdot p_j \quad (1)$$

where r_i denotes positive reward terms with weights w_i , and p_j denotes penalty terms with weights $w_j < 0$. **Only representative terms are shown here; the complete set of weights is provided in Table 3.** The representative individual terms for key locomotion behaviors are defined as follows:

$$r_{\text{vel}} = \exp(-\|v_{xy} - v_{xy}^{\text{cmd}}\|^2 / \sigma) \quad (2)$$

$$r_{\text{gait}} = \prod_k \mathbf{1}[\text{foot}_k \in \text{correct phase}] \quad (3)$$

$$r_{\text{clearance}} = \sum_k \max(0, h_k - h_{\min}) \quad (4)$$

$$p_{\text{wall}} = -\min(d_{\text{depth}}) \quad (5)$$

$$p_{\text{slip}} = -\sum_k \|v_{\text{foot},k}\|^2 \quad (6)$$

where v_{xy} is the base planar velocity, v_{xy}^{cmd} is the commanded velocity, σ is the velocity tracking sensitivity parameter, h_k is the foot clearance of foot k , $h_{\min}$ is the

minimum required clearance, d_{depth} is the minimum depth reading, and $v_{\text{foot},k}$ is the foot contact velocity. Full reward weights are listed in Table 3.

Table 3. Low-Level Reward Function Weights

Reward Term	Weight
Air time reward	5.0
Base angular velocity reward	5.0
Base linear velocity reward	5.0
Foot clearance reward	0.5
Open space seeker reward	0.5
Gait reward	12.0
Visual wall penalty	-0.1
Action smoothness penalty	-1.0
Air time variance penalty	-1.0
Base motion penalty	-2.0
Base orientation penalty	-3.0
Foot slip penalty	-0.5
Joint acceleration penalty	-1.0×10^{-4}
Joint position penalty	-0.7
Joint torque penalty	-5.0×10^{-4}
Joint velocity penalty	-1.0×10^{-2}

The low-level policy is trained with a simulation timestep of 0.002 seconds and a control decimation factor of 20, resulting in an effective control frequency of 25 Hz. It is implemented as a multi-layer perceptron (MLP) with the architecture described in Table 4.

Table 4. Neural Network Architecture

Parameter	Value
Network type	MLP (Actor-Critic)
Actor hidden layers	[512, 256, 128]
Critic hidden layers	[512, 256, 128]
Activation function	ELU
Output activation	Linear
Observation normalization	Running mean/std
Weight initialization	Orthogonal

5.2. High-Level Navigation Policy Configuration

To improve sim-to-real robustness and policy generalization, domain randomization techniques can be employed during training to expose the policy to diverse environmental variations [23]. On top of the locomotion controller, a high-level navigation policy is trained to generate navigation-oriented velocity commands. The high-level controller does not directly control robot joints. Instead, it outputs linear and angular velocity commands that are forwarded to the pre-trained low-level locomotion policy. Table 5 summarizes the observation space of the high-level navigation policy.

Table 5. High-Level Policy Observation Space

Observation	Dim	Description
SLAM-estimated position	2	x, y in world frame
SLAM-estimated heading	1	Yaw angle ψ
Goal relative position	2	$\Delta x, \Delta y$ to goal
Goal relative heading	1	Angular difference to goal
Base linear velocity	2	v_x, v_y feedback
Base angular velocity	1	ω_z feedback
Total	9	

The action space of the high-level policy consists of 2-dimensional continuous commands: forward velocity $v_x \in [0.0, 1.0]$ m/s and yaw rate $\omega_z \in [-0.4, 0.4]$ rad/s. These commands are smoothed using an exponential moving average before being forwarded to the low-level controller:

$$a_t^{\text{smooth}} = \alpha \cdot a_t + (1 - \alpha) \cdot a_{t-1}^{\text{smooth}}, \quad \alpha = 0.25 \quad (7)$$

where the smoothing factor $\alpha = 0.25$ was selected based on preliminary tuning trials to balance high-level command responsiveness and low-level gait stability, preventing abrupt velocity step-changes from destabilizing the robot.

The high-level policy uses the same MLP architecture as the low-level policy but with smaller hidden layers [256, 128] due to the reduced observation dimensionality. Full PPO hyperparameters are listed in Table 6.

Table 6. High-Level PPO Training Parameters

Parameter	Value
Algorithm	PPO
Policy type	MLP Policy
Learning rate	3×10^{-4}
Batch size	128
Rollout steps	256
Epochs per update	4
Discount factor (γ)	0.99
GAE parameter (λ)	0.95
Clip range	0.2
Entropy coefficient	0.01
Value loss coefficient	0.5
Total training timesteps	500,000
High-level decimation	2
Command smoothing factor	0.25
Forward velocity range	0.0 – 1.0 m/s
Yaw rate range	−0.4 – 0.4 rad/s
Maximum episode steps	400

The high-level policy is trained for 500,000 timesteps using hierarchical control. During training, the low-level locomotion policy remains fixed while the high-level policy learns navigation behaviors such as obstacle avoidance, directional control, and long-range movement planning.

Table 7. System resource load and processing speed across configurations over five trials. Values are reported as mean $\pm$ std.

Configuration	Avg. FPS	Min. FPS	RAM Start (GB)	RAM End (GB)	Avg. CPU (%)	Peak CPU (%)
RL Only	10.48 $\pm$ 0.05	8.73 $\pm$ 0.06	18.79 $\pm$ 0.09	18.67 $\pm$ 0.06	16.18 $\pm$ 0.21	25.64 $\pm$ 1.27
RL + VSLAM	9.82 $\pm$ 0.09	8.39 $\pm$ 0.06	19.27 $\pm$ 0.05	19.48 $\pm$ 0.03	19.94 $\pm$ 0.33	26.80 $\pm$ 0.62
RL + VSLAM + TSDF	5.78 $\pm$ 0.21	1.57 $\pm$ 0.19	20.94 $\pm$ 0.14	29.40 $\pm$ 1.53	29.59 $\pm$ 1.32	43.66 $\pm$ 4.68

Table 8. Disk I/O load across configurations over five trials. Values are reported as mean $\pm$ std in MB/s.

Configuration	Avg. Read	Avg. Write	Peak Write
RL Only	0.10 $\pm$ 0.14	0.07 $\pm$ 0.05	3.98 $\pm$ 6.60
RL + VSLAM	0.33 $\pm$ 0.45	0.07 $\pm$ 0.04	3.03 $\pm$ 4.56
RL + VSLAM + TSDF	0.12 $\pm$ 0.12	1.42 $\pm$ 0.20	118.12 $\pm$ 7.68

6. RESULTS

This section evaluates the performance of the proposed system under different indoor navigation scenarios in the Isaac Lab simulation environment, focusing on system-level behavior rather than individual module performance.

6.1. Module Load Impact

To characterize the computational resource contention caused by multi-module integration, we evaluated the system under three configurations: (1) RL locomotion only, (2) RL + VSLAM, and (3) the fully integrated system (RL + VSLAM + Dense Mapping). Five trials were conducted for each configuration. Tables 7 and 8 report mean and standard deviation values across trials, while Figs. 3 and 4 show representative runtime logs for the integrated configurations to illustrate transient performance fluctuations.

RL Locomotion Baseline: As shown by the RL-only baseline in Table 7, the standalone reinforcement learning policy operates with high stability and minimal resource overhead. The CPU usage remains consistently low (fluctuating between 15% and 20%), and the RAM consumption is stable at approximately 18.7 GB to 18.8 GB. Consequently, the system processing speed is maintained at approximately 10.5 FPS throughout the simulation, with minimal disk I/O load.

Table 7 summarizes the system resource load across five trials for each configuration. The RL-only baseline maintained the highest processing speed, with an average FPS of 10.48 ± 0.05 and stable memory usage around 18.7–18.8 GB. Adding VSLAM slightly reduced the average FPS to 9.82 ± 0.09 and increased average CPU usage from $16.18 \pm 0.21\%$ to $19.94 \pm 0.33\%$, indicating additional computational load from visual feature extraction and pose estimation.

The fully integrated RL + VSLAM + TSDF configuration produced the most substantial system load. Average FPS decreased to 5.78 ± 0.21 , while the minimum FPS dropped to 1.57 ± 0.19 . RAM usage increased from

20.94 ± 0.14 GB at the beginning of the run to 29.40 ± 1.53 GB by the end, reflecting the memory growth introduced by TSDF-based dense reconstruction. As shown in Table 8, the TSDF configuration also introduced much higher disk write activity, with an average write rate of 1.42 ± 0.20 MB/s and peak write rate of 118.12 ± 7.68 MB/s. These results indicate that dense mapping acts as a major observed computational and memory bottleneck in this simulation setup, and that its concurrent execution is associated with reduced control-loop throughput.

Impact of Visual SLAM: When ORB-SLAM3 is introduced (Fig. 3), a noticeable increase in resource consumption is observed. CPU usage exhibits greater variance, occasionally peaking near 28%, driven by continuous visual feature extraction and pose optimization tasks. RAM usage shows a step increase, stabilizing around 19.4 GB. Additionally, the disk I/O reveals occasional write spikes, likely corresponding to keyframe logging or map management. The system processing speed experiences slight instability, fluctuating between 9.5 and 10.25 FPS, indicating early signs of resource contention between the perception and control loops.

Impact of Dense Mapping: In the representative runtime log shown in Fig. 4, the integration of the TSDF-based dense mapping pipeline imposes a substantial computational burden. CPU usage becomes highly erratic, frequently oscillating between 25% and 40%. More critically, the RAM usage exhibits a steady, monotonic increase from 21 GB to over 27 GB by the end of the simulation. This linear accumulation is characteristic of volumetric dense mapping, where the environment representation continuously expands as new depth data is integrated.

In this representative run, resource contention decreases the overall system throughput. Specifically, the continuous accumulation and saving of the dense volumetric map require frequent disk write operations. This I/O burden may reduce CPU execution efficiency, as compute threads can stall while waiting for disk operations to complete. Consequently, the processing speed drops from an initial 8.5 FPS to approximately 4.5 FPS, with transient drops below 3 FPS. These FPS drops can degrade the stability of the control loop used by the RL locomotion policy and are associated with unstable gaits and reduced navigation performance in more complex environments.

This degradation suggests increased latency in the pose and observation updates received by the high-level RL



Fig. 3. System performance logs for the combined RL and Visual SLAM modules. The introduction of ORB-SLAM3 increases CPU variance and causes minor fluctuations in processing speed.

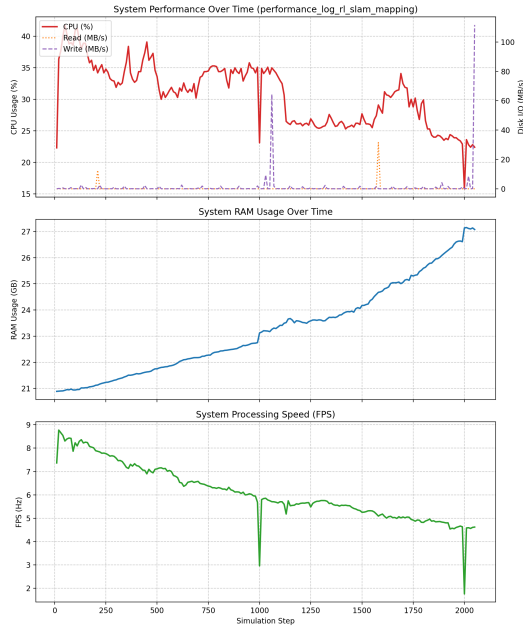


Fig. 4. System performance logs for the fully integrated navigation system (RL + VSLAM + Dense Mapping). The TSDF volumetric reconstruction leads to linear RAM accumulation and noticeable FPS degradation over time.

policy. Such delayed and intermittent observations may reduce the consistency of the policy input distribution relative to training conditions. This effect is consistent with the observed oscillatory velocity commands and re-

duced goal-reaching performance, although the performance drop in Map C also reflects the combined effects of narrow corridor geometry, reactive control without a global planner, and limited narrow-space training diversity.

6.2. Navigation Success Rate

Table 9 summarizes the average navigation performance across three indoor environments with different levels of structural complexity. While five trials were conducted for each environment to calculate the average success rate and collision metrics, the trajectory and position error analyses presented in this section are extracted from a single representative trial per environment. This approach allows for a clearer examination of specific SLAM tracking behaviors and control dynamics.

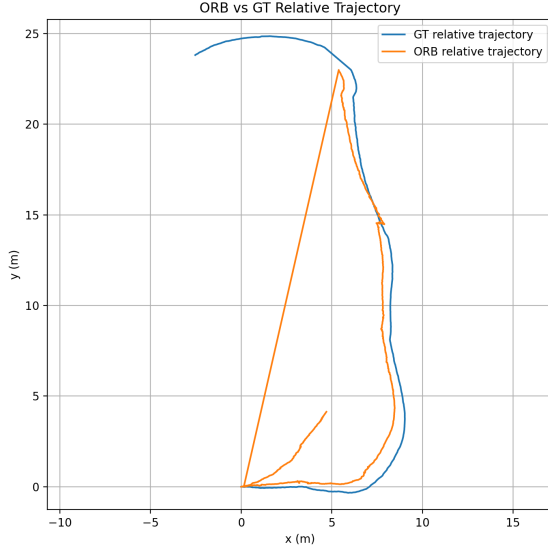
Map A (Room): Map A represents a relatively simple indoor room with large open spaces and minimal obstacles, where the system achieved an 80% success rate. As shown in the relative trajectory (Fig. 5) and position error (Fig. 6), the open and textureless nature of the environment poses a severe challenge for visual tracking. When traversing feature-poor regions, ORB-SLAM3 frequently loses tracking, causing the estimated pose to temporarily reset or jump (evident from the sudden error spike to nearly 25 m). Despite this large localization discontinuity, the high-level policy was often able to recover and guide the robot to the goal. The single failure case occurred when the robot ventured too deep into the featureless center of the room, leading to an irrecoverable tracking loss that prevented it from reaching the destination. Furthermore, the large standard deviation in the average steps taken (1539.99 ± 740.63) indicates that system performance is highly sensitive to the exact timing and physical location of SLAM tracking losses, resulting in significant variance across different trials.

Map B (Warehouse): Map B corresponds to a warehouse-like environment containing moderate obstacles and shelf structures. Here, the robot successfully reached the goal in all trials (100% success rate). As depicted in Fig. 7 and Fig. 8, the SLAM-estimated trajectory closely follows the ground truth. While the position error gradually accumulates to approximately 1.75 m due to natural visual odometry drift, the system maintains continuous tracking without major tracking loss. Although minor collisions occasionally occurred when passing close to shelf structures or narrow gaps, the integrated system demonstrated relatively stable locomotion in moderately cluttered spaces.

Map C (Narrow Corridor): Map C is a self-designed narrow corridor environment with dense obstacles and constrained traversable space (4–5 m in width). Rather than a standard navigation task, this map serves as an out-of-distribution stress test to reveal the operational boundaries of the integrated system. The system achieved a 0%

Table 9. Average navigation performance across different environments (5 trials each). Failure cases are recorded as 3000 steps (episode timeout).

Environment	Success Rate (%)	Avg. Collisions (mean $\pm$ std)	Avg. Steps (mean $\pm$ std)	Avg. Time (s) (mean $\pm$ std)
Map A (Room)	80	0.00 $\pm$ 0.00	1539.99 $\pm$ 740.63	118.74 $\pm$ 8.22
Map B (Warehouse)	100	0.40 $\pm$ 0.55	1227.60 $\pm$ 125.45	127.08 $\pm$ 11.91
Map C (Corridor)	0	0.40 $\pm$ 0.55	3000.00 (Timeout)	—

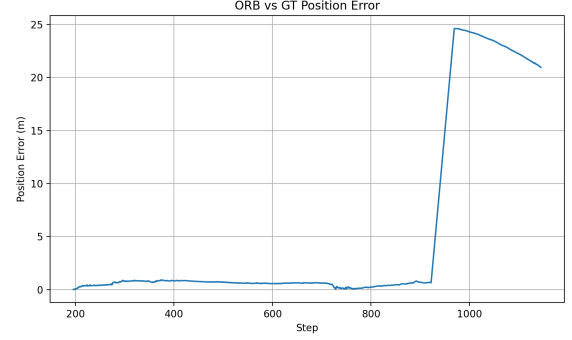
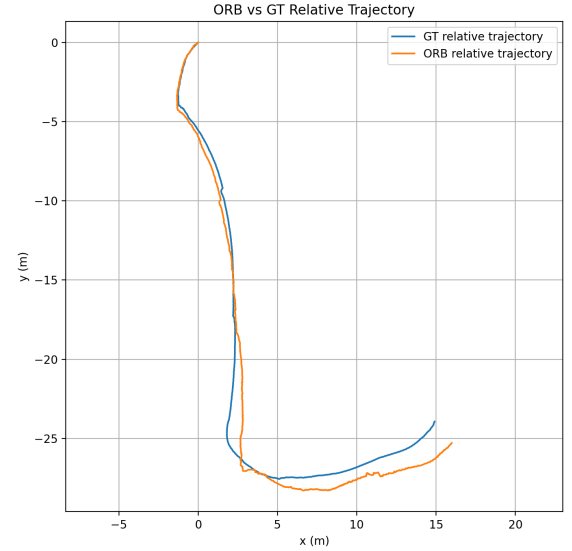
**Fig. 5.** Relative trajectory comparison (ORB vs. GT) in Map A (Room). The sudden trajectory deviation illustrates a severe tracking loss event due to featureless regions.

success rate in this scenario, indicating a combined failure mode rather than a failure of a single module. As visualized in the looping patterns of the relative trajectory (Fig. 9), the narrow corridor geometry differs from the main training distribution and is difficult for the purely reactive high-level RL policy, which does not include a global planner. This limitation is further exacerbated by delayed SLAM pose updates and FPS degradation under full system load, causing local obstacle avoidance behavior to converge into circular motion.

6.3. Dense Mapping Results

To evaluate the effect of pose quality on dense reconstruction, we reconstruct the environment using the TSDF pipeline [19] under two pose input conditions: (1) simulator ground-truth poses and (2) ORB-SLAM3 estimated poses.

As shown in Fig. 11, the ground-truth-based reconstruction produces a geometrically consistent and complete map of the environment. In contrast, when SLAM-estimated poses are used, the reconstructed map exhibits noticeable distortion and structural misalignment due to accumulated drift and tracking uncertainty.

**Fig. 6.** Position error over time in Map A. The sharp spike corresponds to the ORB-SLAM3 tracking failure and subsequent pose reset.**Fig. 7.** Relative trajectory comparison (ORB vs. GT) in Map B (Warehouse). The system maintains continuous tracking despite structural clutter.

Specifically, the SLAM-estimated pose map exhibits structural misalignment primarily in regions where ORB-SLAM3 experienced tracking instability or accumulated drift, consistent with the position error patterns reported in Section 6.2. This comparison suggests that pose drift, such as the approximately 1.75 m error observed in Map B, can lead to visually noticeable geometric distortion in volumetric reconstruction. These results indicate that pose correction mechanisms may be important when TSDF map-

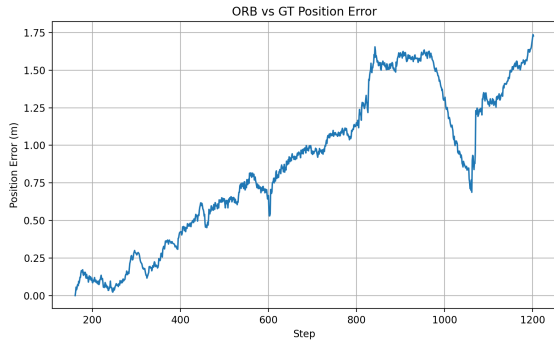


Fig. 8. Position error over time in Map B. The error gradually accumulates due to odometry drift but avoids catastrophic tracking failures.

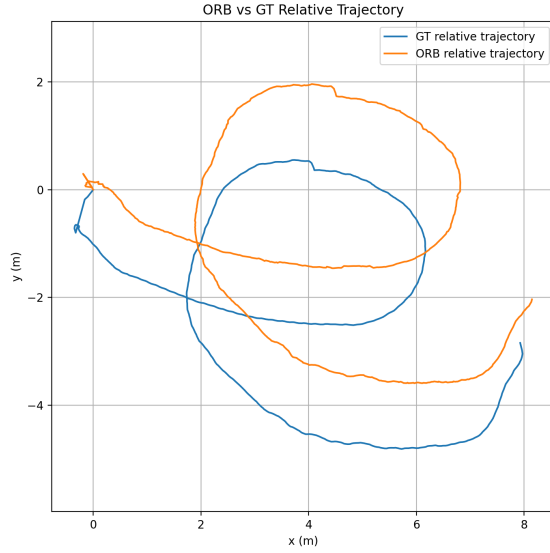


Fig. 9. Relative trajectory comparison (ORB vs. GT) in Map C (Narrow Corridor). The looping pattern reflects a combined effect of narrow geometry, reactive control, delayed pose updates, and reduced system throughput.

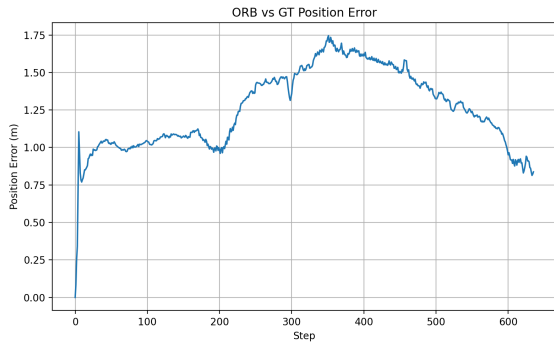
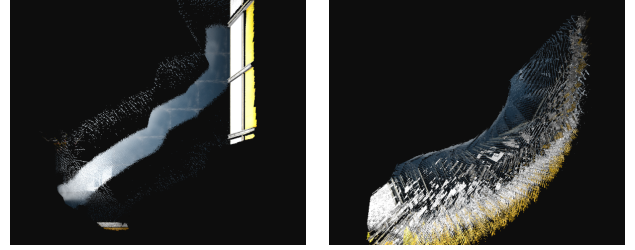


Fig. 10. Position error over time in Map C. The fluctuating error reflects the robot's circular motions before ultimately failing the navigation task.



(a) Ground-truth pose (b) ORB-SLAM3 estimated pose

Fig. 11. Comparison of Open3D TSDF-based dense reconstruction using ground-truth poses and ORB-SLAM3 estimated poses. The SLAM-based reconstruction shows visible distortion caused by pose drift and tracking instability.

ping is used together with SLAM-estimated poses, especially under computational constraints or visually challenging environments.

7. FAILURE ANALYSIS

Although the proposed framework demonstrates stable navigation performance in structured indoor environments, several failure cases were consistently observed during experiments. These failures primarily originate from the interaction between visual perception, reinforcement learning-based control, and real-time system constraints. Table 10 summarizes the key failure modes identified during the evaluation, detailing their triggers, underlying causes, and potential mitigation strategies.

One major failure mode occurs when the robot enters textureless environments with insufficient visual features. In such situations, ORB-SLAM3 may lose tracking temporarily, leading to unstable pose estimation and navigation failure. This issue was particularly observed in large open spaces with limited geometric structure.

A second failure mode was repetitive avoidance behavior in narrow corridor environments. The reinforcement learning policy was primarily trained in relatively wide environments, and the narrow corridor geometry therefore acted as an out-of-distribution condition for the reactive high-level controller. Together with delayed pose updates and reduced system throughput, this caused the robot to repeatedly execute local obstacle avoidance maneuvers and spin in circles, preventing forward progress toward the goal.

A third failure mode involved obstacle collisions in cluttered environments, primarily caused by the delayed perception-response loop between depth sensing and locomotion control. When the robot moved through densely arranged obstacles, the control system could not react in time to avoid contact, resulting in physical collisions with

Table 10. Observed failure modes during navigation experiments.

Failure Type	When It Happens	Likely Cause	Effect on System	Possible Fix
Tracking Loss	Textureless open areas	Insufficient visual features	Localization failure	Feature enhancement or sensor fusion
Circular Avoidance Behavior	Narrow corridors	Unseen environment geometry	Navigation instability	More diverse RL training
Obstacle Collision	Cluttered environments	Delayed perception-response loop	Physical collision	Lower latency control
Dense Map Distortion	SLAM-based reconstruction	Accumulated pose drift	Geometric inconsistency	Pose correction or loop closure
Hopping Gait	High-frequency terrain variation	Reward shaping insufficient for gait stability	Energy inefficiency, reduced speed	Gait regularization reward tuning

nearby structures.

A fourth failure mode was observed in dense reconstruction when using SLAM-estimated poses. Accumulated localization drift caused noticeable geometric distortion and structural misalignment in the reconstructed map, suggesting that TSDF mapping quality is sensitive to pose estimation stability.

A fifth failure mode involved hopping gait instability, observed primarily during locomotion over high-frequency terrain variations. When the gait reward shaping was insufficient to enforce consistent stepping timing, the robot exhibited irregular hopping patterns that reduced locomotion efficiency and forward speed, occasionally causing balance loss on uneven surfaces.

These results indicate that perception robustness, environment diversity during training, and real-time synchronization are critical factors for reliable integrated robotic navigation systems.

8. DISCUSSION

The experimental results demonstrate that integrating visual SLAM, reinforcement learning-based locomotion, and dense mapping into a unified robotic framework introduces significant system-level challenges beyond the performance of individual modules.

First, the results show that navigation performance strongly depends on the interaction between perception stability and learned control policies. Although reinforcement learning enables adaptive locomotion behavior, the policy remains highly sensitive to environmental distributions not observed during training. This limitation became evident in the narrow corridor environment (Map C), where the system exhibited poor goal-reaching behavior under an out-of-distribution narrow-corridor condition. Because the policy was primarily optimized for unstructured, obstacle-rich disaster scenarios rather than highly structured, symmetric passages, the narrow corridor effec-

tively acted as an out-of-distribution environment. Lacking the spatial topological understanding of a global planner, the reactive RL policy tended to converge to local avoidance behavior, resulting in circular motion.

Second, the experiments highlight the importance of reliable pose estimation for downstream modules. Dense reconstruction quality degraded noticeably when SLAM-estimated poses were used instead of simulator ground-truth poses. This suggests that mapping performance in practical robotic systems can be strongly affected by localization robustness.

Furthermore, the integration of multiple computationally intensive modules under ROS2 introduced resource contention and synchronization delays, reducing overall system stability. Even when each module performed adequately in isolation, simultaneous execution often produced unstable behavior due to temporal inconsistency and communication latency.

Several limitations remain in the current work. All experiments were conducted in simulation, and the reinforcement learning policy was trained under relatively limited environment diversity. In addition, dense mapping was evaluated primarily in a decoupled configuration using ground-truth poses.

To further address these bottlenecks, future integrated navigation systems may benefit from tighter coupling between visual SLAM and dense mapping modules. Currently, ORB-SLAM3 and TSDF-based mapping independently process visual information, leading to duplicated feature extraction and additional computational overhead. A more unified framework that reuses ORB-SLAM3 feature representations for dense reconstruction may reduce CPU and memory consumption while improving overall system efficiency under resource-constrained conditions.

Moreover, the experimental results indicate that reinforcement learning policies remain highly sensitive to environmental distributions encountered during training.

Therefore, future training strategies should incorporate a wider variety of indoor geometries, including narrow corridors, cluttered rooms, and irregular obstacle layouts, in order to improve generalization capability across diverse indoor navigation scenarios.

Future work will focus on improving environment generalization, incorporating more robust sensor fusion methods, and deploying the proposed framework on real quadruped robotic platforms.

9. CONCLUSION

This paper presented a ROS2-based integrated navigation framework combining visual SLAM, reinforcement learning-based locomotion, and dense mapping for quadruped robots in simulated indoor environments.

This work investigated observed coupling effects under concurrent execution, including the association between delayed and intermittent SLAM pose updates and RL-based navigation performance, the memory overhead introduced by TSDF-based dense mapping, and the environment-dependent limitations of the locomotion policy in narrow corridor settings. These observations provide a system-level view of practical integration challenges that may not be apparent when perception, control, and mapping modules are evaluated separately.

Additional analysis of dense reconstruction further showed that mapping quality is sensitive to the accuracy of pose estimation, particularly when SLAM drift accumulates during long-term navigation.

Overall, this study provides practical insights into the deployment limitations and integration challenges of vision-based quadruped robotic systems, highlighting the importance of robust perception, environment diversity, and efficient multi-module coordination for future autonomous navigation systems.

DECLARATIONS

Funding

This work was supported by the National Science and Technology Council (NSTC) Undergraduate Research Project of Taiwan.

Conflict of Interest

The authors declare that there is no conflict of interest regarding the publication of this paper.

REFERENCES

- [1] K. Nagatani et al., “Emergency response to the nuclear accident at the Fukushima Daiichi Nuclear Power Plants using mobile rescue robots,” *Journal of Field Robotics*, vol. 30, no. 1, pp. 44-63, 2013.
- [2] M. Hutter et al., “ANYmal - a highly mobile and dynamic quadrupedal robot,” in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Daejeon, Korea, pp. 38-44, 2016.
- [3] J. Lee, J. Hwangbo, L. Wellhausen et al., “Learning quadrupedal locomotion over challenging terrain,” *Science Robotics*, vol. 5, no. 47, 2020.
- [4] T. Miki, J. Lee, J. Hwangbo et al., “Learning robust perceptive locomotion for quadrupedal robots in the wild,” *Science Robotics*, vol. 7, no. 62, 2022.
- [5] L. Tai, G. Paolo and M. Liu, “Virtual-to-real deep reinforcement learning: Continuous control of mobile robots for mapless navigation,” in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 31-36, 2017.
- [6] C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. M. Montiel and J. D. Tardós, “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multimap SLAM,” *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874-1890, Dec. 2021.
- [7] J. Cadena et al., “Past, present, and future of simultaneous localization and mapping,” *IEEE Transactions on Robotics*, vol. 32, no. 6, pp. 1309-1332, 2016.
- [8] J. Schulman et al., “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [9] N. Heess et al., “Emergence of locomotion behaviours in rich environments,” *arXiv preprint arXiv:1707.02286*, 2017.
- [10] A. Kumar, Z. Fu, D. Pathak and J. Malik, “RMA: Rapid motor adaptation for legged robots,” in *Proc. Robotics: Science and Systems (RSS)*, 2021.
- [11] C. San Vicente Gutiérrez et al., “Towards a distributed and real-time framework for robots: Evaluation of ROS 2.0 communications for real-time robotic applications,” *arXiv preprint arXiv:1809.02595*, 2018.
- [12] S. Macenski et al., “Robot operating system 2: Design, architecture, and uses in the wild,” *Science Robotics*, vol. 7, no. 66, 2022.
- [13] M. Mittal, P. Roth, J. Tigue et al., “Isaac Lab: A GPU-Accelerated Simulation Framework for Multi-Modal Robot Learning,” *arXiv preprint arXiv:2511.04831*, 2025.
- [14] F. Xia et al., “Gibson Env: Real-world perception for embodied agents,” in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 9068-9079, 2018.
- [15] T. Whelan, S. Leutenegger, R. F. Salas-Moreno, B. Glocker and A. J. Davison, “ElasticFusion: Dense SLAM without a pose graph,” in *Proc. Robotics: Science and Systems (RSS)*, Rome, Italy, 2015.
- [16] M. Labbé and F. Michaud, “RTAB-Map as an open-source lidar and visual simultaneous localization and mapping library for large-scale and long-term online operation,” *Journal of Field Robotics*, vol. 36, no. 2, pp. 416-446, 2019.
- [17] D. Chaplot, D. Gandhi, S. Gupta et al., “Learning to explore using active neural SLAM,” in *Proc. International Conference on Learning Representations (ICLR)*, 2020.

- [18] M. Savva et al., “Habitat: A platform for embodied AI research,” in *Proc. IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 9338-9346, 2019.
- [19] M. Fehr et al., “TSDF-based change detection for consistent long-term dense reconstruction and dynamic object discovery,” in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, Singapore, pp. 5237-5244, 2017.
- [20] R. A. Newcombe et al., “KinectFusion: Real-time dense surface mapping and tracking,” in *Proc. IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, pp. 127-136, 2011.
- [21] Y. Maruyama, S. Kato and T. Azumi, “Exploring the performance of ROS2,” in *Proc. International Conference on Embedded Software (EMSOFT)*, pp. 1-10, 2016.
- [22] V. Makoviychuk et al., “Isaac Gym: High performance GPU-based physics simulation for robot learning,” *arXiv preprint arXiv:2108.10470*, 2021.
- [23] J. Tobin et al., “Domain randomization for transferring deep neural networks from simulation to the real world,” in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 23-30, 2017.
- [24] N. Rudin, D. Hoeller, P. Reist and M. Hutter, “Learning to walk in minutes using massively parallel deep reinforcement learning,” in *Proc. Conference on Robot Learning (CoRL)*, 2021.



You-Hao Li is currently pursuing the B.S. degree in Artificial Intelligence at Tamkang University, New Taipei City, Taiwan. His research interests include artificial intelligence speech systems, reinforcement learning, embedded systems, robotics, and autonomous systems.



Jaesik Jeong received the Ph.D. degree in Electrical Engineering from National Taiwan Normal University, Taipei, Taiwan. He is currently an Assistant Professor with the Department of Artificial Intelligence, Tamkang University, Taiwan. His research interests include robotics, computer vision, artificial intelligence, and autonomous systems.

Publisher’s Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.